

TP 1 SUR 4

binggge

Un suivi de séries • Node + React • PostgreSQL • GitLab

Une appli minuscule, déployée comme une vraie.

CE QU'ELLE FAIT

Chercher une série, l'ajouter à sa liste, cocher les épisodes vus.

AVEC QUOI

API Node dans `api/`, page React dans `web/`, PostgreSQL en conteneur. Un dépôt.

CE QUI EST NOTÉ

Pas l'appli. La façon dont elle arrive en production.

01

01 / 07 • 8 MIN

Créez le dépôt sur GitLab

Projet nommé binggge, sans README, sans licence. Ajoutez-moi en Reporter.

Vide, vraiment vide. Un README généré par GitLab vous obligerait à un premier pull inutile.

Une clé dédiée à ce cours

```
ssh-keygen -t ed25519 -C "binggge" \  
    -f ~/.ssh/id_binggge  
# -f impose le nom du fichier :  
# sans lui vous écrasez une clé existante  
cat ~/.ssh/id_binggge.pub  
# à coller dans GitLab, Préférences, Clés SSH  
ssh -T git@gitlab.com
```

02

02 / 07 • 7 MIN

Verrouillez main

Réglages, Dépôt, Branches protégées. Sur main : autorisés à pousser, personne. Autorisés à merger, les mainteneurs.

Puis essayez un push direct. Il doit être refusé. C'est le but de l'étape.

```
git push origin main
remote: GitLab: You are not allowed
to push code to protected branches
```

03

03 / 07 • 12 MIN

Le dépôt local, et la branche

Linux / macOS • sur Windows, dans Git Bash

```
mkdir binggge && cd binggge && git init -b main
git config user.name "Prénom Nom"
printf 'node_modules\n.env\n' > .gitignore
printf '* text=auto eol=lf\n' > .gitattributes
git remote add origin git@gitlab.com:VOUS/binggge.git
git switch -c feat/squelette
```

04

04 / 07 • 30 MIN

Trois routes qui répondent

GET /health

Renvoie { status: "ok" }.
Trois lignes. C'est elle que le serveur interrogera en séance 3.

GET /shows?q=

Appelle TVMaze et renvoie une liste allégée : identifiant, titre, année, image.

GET /watchlist

Renvoie un tableau vide. La base arrive en séance 2, la route existe dès maintenant.

RÉFÉRENCE · AUCUNE CLÉ, AUCUN COMPTE

L'API TVMaze

Rechercher	<code>api.tvmaze.com/search/shows?q=severance</code>
Une série	<code>api.tvmaze.com/shows/44778</code>
Ses épisodes	<code>api.tvmaze.com/shows/44778/episodes</code>

Filtrez la réponse avant de la renvoyer à votre front. Ne transmettez jamais tel quel ce qu'une API tierce vous donne.

05-06

05 / 07 • 18 MIN

Poussez, puis ouvrez la merge request

```
git add .  
git commit -m "feat: squelette api et web"  
git push -u origin feat/squelette  
# GitLab renvoie le lien de la MR
```

La description tient en trois lignes :

Ce que ça fait.

Ce que ça ne fait pas encore.

Comment le tester en une commande.

Vous êtes aussi le chef de projet

Personne ne vous donnera de tickets. Vous écrivez les vôtres, aujourd'hui, pour les trois séances qui restent.

Dans GitLab, Suivi puis Tickets : huit tickets minimum, un par chose à faire. Trois jalons, un par séance restante.

Chaque ticket porte une estimation en heures. Elle sera fausse. Vous la comparerez au réel en séance 4.

Un tableau à trois colonnes suffit : à faire, en cours, fait. Une seule carte en « en cours » à la fois.

Un ticket bien écrit

Titre : POST /watchlist refuse un titre vide

Jalon : Séance 2

Estimation : /estimate 1h

Et sa MR le referme toute seule

```
git commit -m "fix: refuse un titre vide
```

```
Closes #3"
```

Closes #3 dans la description de la MR ferme le ticket au merge.
Plus rien à cocher à la main.

Écrivez la suite avant de la coder

Un README qu'un inconnu peut suivre, et le squelette des tests de la séance 2. Les tests doivent échouer : c'est le but.

LE README

Ce que fait binggge. Comment le démarrer en deux commandes. Les routes existantes. Ce qui n'existe pas encore.

api/tests/server.test.js

```
test.todo('inscription puis connexion')
test.todo('watchlist sans en-tête : 401')
test.todo('un titre vide est refusé')
# npm test les affiche en attente
```

Échangez votre README avec votre voisin. S'il ne démarre pas le projet sans vous parler, réécrivez-le.

Avant de fermer la machine

01

Le dépôt binggge existe et je suis Reporter dessus.

02

Un push direct sur main est refusé.

04

`curl localhost:3000/shows?q=severance`
renvoie des séries.

05

Huit tickets, trois jalons, une estimation sur chacun. Et un README lisible.

03

Une MR décrite en trois lignes, ouverte puis mergée.

Séance 2 : authentication, PostgreSQL, et beaucoup de tests.